

**Title of Topic: Relational Algebra, Domain Relational Calculus, Tuple
Relational Calculus**

Course Code: 23IT201

Course Title: DataBase Management Systems

Session Number: 4

Academic Year: 2024 – 2025 (Even Sem)

Module 1 – Data Modeling and Relational Query Language

- 1. Introduction to DBMS, Characteristics of DBMS, DBMS vs File Systems, need for DBMS.**
- 2. Three Level DBMS Architecture**
- 3. Data Models Introduction, Benefits, and Phases**
- 4. ER Diagrams – Symbols, Components, Relationships, Weak entities, Attributes, Cardinality.**
- 5. Relational Algebra, Domain Relational Calculus, Tuple Relational**
- 6. Calculus Keys - primary Key, Foreign Key.**

Relational Algebra, Domain Relational Calculus, Tuple Relational

Calculus

Topics to be Covered

- Relational Algebra
- Schema for the University Database
- Select Operation (σ)
- Project Operation (π)
- Union Operation ($\cup$)
- Set Difference Operation ($-$)
- Set-Intersection Operation ($\cap$)
- Cartesian-Product Operation ($\times$)
- Rename Operation (ρ)
- Natural-Join Operation
- Assignment Operation
- Generalized Projection
- Aggregate Functions
- Tuple Relational Calculus
- Domain Relational Calculus

Relational Algebra

- Procedural language
- Six basic operators
 - ✓ select: σ
 - ✓ project: Π
 - ✓ union: $\cup$
 - ✓ set difference: $-$
 - ✓ Cartesian product: $\times$
 - ✓ rename: ρ
- The operators take one or two relations as inputs and produce a new relation as a result
- Other operations are set intersection, natural join, and assignment

Schema for the University DataBase

- Instructor(ID, name, dept_name ,salary)
- Teaches(ID ,course_id ,sec_id, semester, year)
- Takes(ID ,course_id ,sec_id,semester, year,grade)
- Section(course_id, sec_id,semester,year,building,room_no,time_slot_id)
- Student(ID,name,dept_name,tot_cred)
- Classroom(building, room_no,capacity)
- Department(dept_name,building,Budget)
- Course(course_id, title,dept_name,credits)

Relational Algebra: Part-I:



Part-II:



Part-III:



Select Operation (σ)

- The select operation selects tuples that satisfy a given predicate.
- Notation: $\sigma_p(r)$, where p is called the selection predicate
- Defined as: $\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$

Where p is a formula in propositional calculus consisting of terms connected by : $\wedge$ (and), $\vee$ (or), $\neg$ (not)

- Each term is one of: <attribute> op <attribute> or <constant>
- where op is one of: $=$, $\neq$, $>$, $\geq$, $<$, $\leq$
- E.g., $\sigma_{\text{dept_name}=\text{"Physics"}}(\text{instructor})$

Select Operation (σ)

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

The instructor relation

ID	name	dept_name	salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

$\sigma_{dept_name="Physics"}(instructor)$

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

$\sigma_{dept_name="Physics"}(instructor)$

ID	name	dept_name	salary
22222	Einstein	Physics	95000

$\sigma_{dept_name="Physics" \wedge salary > 90000}(instructor)$

Project Operation (Π)

- The project operation is a unary operation that returns its argument relation, with certain attributes left out. Since a relation is a set, any duplicate rows are eliminated
- Notation: $\Pi_{A_1, A_2, A_3}(r)$, where A_1, A_2 are attribute names and r is a relation name
- The result is defined as the relation of k columns obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets
- E.g., To eliminate the dept_name attribute of instructor
- $\Pi_{ID, name, salary}(\text{instructor})$

Project Operation (Π)

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

The instructor relation

<i>ID</i>	<i>name</i>	<i>salary</i>
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

$\Pi_{ID, name, salary}$ (instructor)

Union Operation ($\cup$)

- Ensure that unions are taken between compatible relations
- Notation: $r \cup s$
- Defined as: $r \cup s = \{t \mid t \in r \text{ or } t \in s\}$
- For $r \cup s$ to be valid:
 - ✓ r, s must have the same arity (same number of attributes)
 - ✓ The attribute domains must be compatible (E.g., 2nd column of r deals with the same type of values as does the 2nd column of s)
- E.g., To find all courses taught in the Fall 2009 semester, or in the Spring 2010 semester, or in both

$$\Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2009} (\text{section})) \cup \Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2010} (\text{section}))$$

Union Operation ($\cup$)

<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>building</i>	<i>room_number</i>	<i>time_slot_id</i>
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

The section relation

Union Operation (U)

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

$\Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2009} (\text{section})) \cup \Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2010} (\text{section}))$

Set Difference Operation (-)

- Allows us to find tuples that are in one relation but are not in another ($r - s$)
- Defined as: $r - s = \{t \mid t \in r \text{ and } t \notin s\}$
- Set differences must be taken between compatible relations
 - ✓ r and s must have the same arity
 - ✓ attribute domains of r and s must be compatible
- E.g., To find all courses taught in the Fall 2009 semester, but not in the Spring 2010 semester

$\Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2009} (\text{section})) - \Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2010} (\text{section}))$

Set Difference Operation (-)

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

course_id
CS-347
PHY-101

$\Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2009} (\text{section})) - \Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2010} (\text{section}))$

Set-Intersection Operation ($\cap$)

- Used to select common rows (tuples) from two tables (relations)
- Defined as: $r \cap s = \{ t \mid t \in r \text{ and } t \in s \}$
- Assume:
 - ✓ r, s have the same arity
 - ✓ attributes of r and s are compatible
- Note: $r \cap s = r - (r - s)$
- E.g., To find the set of all courses taught in both the Fall 2009 and the Spring 2010 semesters

$\Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2009} (\text{section})) \cap \Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2010} (\text{section}))$

Set-Intersection Operation ($\cap$)

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

course_id

CS-101

$\Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2009} (\text{section})) \cap \Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2010} (\text{section}))$

Cartesian-Product Operation ($\times$)

- Allows to combine information from any two relations
- Defined as: $r \times s = \{t \mid t \in r \text{ and } t \in s\}$
- Assume that attributes of $r(R)$ and $s(S)$ are disjoint. (That is, $R \cap S = \emptyset$)
- If attributes of $r(R)$ and $s(S)$ are not disjoint, then renaming must be used
- n_1 tuples in instructor and n_2 tuples in teaches then there are $n_1 * n_2$ ways of choosing a pair of tuples; so there are $n_1 * n_2$ tuples in $r \times s$
- E.g., Schema is Instructor(ID, name, dept_name, salary) and Teaches(ID, course_id, sec_id, semester, year)
- To find the names of all instructors in the Physics department together with the course id of all courses they taught

$\Pi_{\text{name, courseid}}(\sigma_{\text{instructor.ID=teaches.ID}}(\sigma_{\text{deptname='Physics'}}(\text{instructor} \times \text{teaches})))$

Cartesian-Product Operation (×)

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

The instructor relation

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

The teaches relation

Cartesian-Product Operation (×)

inst.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Physics	95000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Physics	95000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	FIN-201	1	Spring	2010
10101	Srinivasan	Physics	95000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Physics	95000	10101	CS-101	1	Fall	2009
12121	Wu	Physics	95000	10101	CS-315	1	Spring	2010
12121	Wu	Physics	95000	10101	CS-347	1	Fall	2009
12121	Wu	Physics	95000	10101	FIN-201	1	Spring	2010
12121	Wu	Physics	95000	15151	MU-199	1	Spring	2010
12121	Wu	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Physics	95000	10101	CS-101	1	Fall	2009
15151	Mozart	Physics	95000	10101	CS-315	1	Spring	2010
15151	Mozart	Physics	95000	10101	CS-347	1	Fall	2009
15151	Mozart	Physics	95000	10101	FIN-201	1	Spring	2010
15151	Mozart	Physics	95000	15151	MU-199	1	Spring	2010
15151	Mozart	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2009
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2010
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2009
22222	Einstein	Physics	95000	10101	FIN-201	1	Spring	2010
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

r = instructor × teaches is: (instructor.ID, instructor.name, instructor.dept_name, instructor.salary, teaches.ID, teaches.course_id, teaches.sec_id, teaches.semester, teaches.year)

Rename Operation (ρ)

- Used to rename a relation or an attribute or an algebraic expression of a relation and is denoted by the lowercase Greek letter ρ rho ()
- E.g., $\rho_x(E)$ returns the expression E under the name X
- If a relational-algebra expression E has arity n, then

$$\rho_{x(A_1, A_2, \dots, A_n)}(E)$$

returns the result of expression E under the name X, and with the attributes renamed to $A_1, A_2, \dots, A_n$

- To find the highest salary in the university with schema Instructor(ID, name, dept_name, salary)

Rename Operation (ρ)

ID	name	dept name	salary	d.ID	d.name	d.dept name	d.salary
10101	Srinivasan	Comp. Sci.	65000	10101	Srinivasan	Comp. Sci.	65000
10101	Srinivasan	Comp. Sci.	65000	12121	Wu	Finance	90000
10101	Srinivasan	Comp. Sci.	65000	22222	Einstein	Physics	95000
12121	Wu	Finance	90000	10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000	12121	Wu	Finance	90000
12121	Wu	Finance	90000	22222	Einstein	Physics	95000
22222	Einstein	Physics	95000	10101	Srinivasan	Comp. Sci.	65000
22222	Einstein	Physics	95000	12121	Wu	Finance	90000
22222	Einstein	Physics	95000	22222	Einstein	Physics	95000

Instructor X d.instructor

- Salary that are not the largest

$\pi_{\text{instructor.salary}} (\sigma_{\text{instructor.salary} < \text{d.salary}} (\text{instructor} \times \rho_d (\text{instructor})))$

Salary
65000
65000
90000

Rename Operation (ρ)

Salary
95000

- Largest Salary
- $\pi_{\text{salary}}(\text{instructor}) - \pi_{\text{instructor.salary}}(\sigma_{\text{instructor.salary} < \text{d.salary}}(\text{instructor} \times \rho_d(\text{instructor})))$
- Compute first a temporary relation consisting of those salaries that are not the largest and
 - Take the set difference between the relation salary (instructor) and the temporary relation just computed, to obtain the result
 - To fetch customer names from customer relation and to rename the resulted relation to CUST_NAMES

Customer_Id	Customer_Name	Customer_City
-----	-----	-----
C10100	Steve	Agra
C10111	Raghu	Agra
C10115	Chaitanya	Noida
C10117	Ajeet	Delhi
C10118	Carl	Delhi

The customer relation

Rename Operation (ρ)

```
 $\rho(\text{CUST\_NAMES}, \Pi(\text{Customer\_Name})(\text{CUSTOMER}))$ 
```

Output:

```
CUST_NAMES
```

```
-----
```

```
Steve
```

```
Raghu
```

```
Chaitanya
```

```
Ajeet
```

```
Carl
```

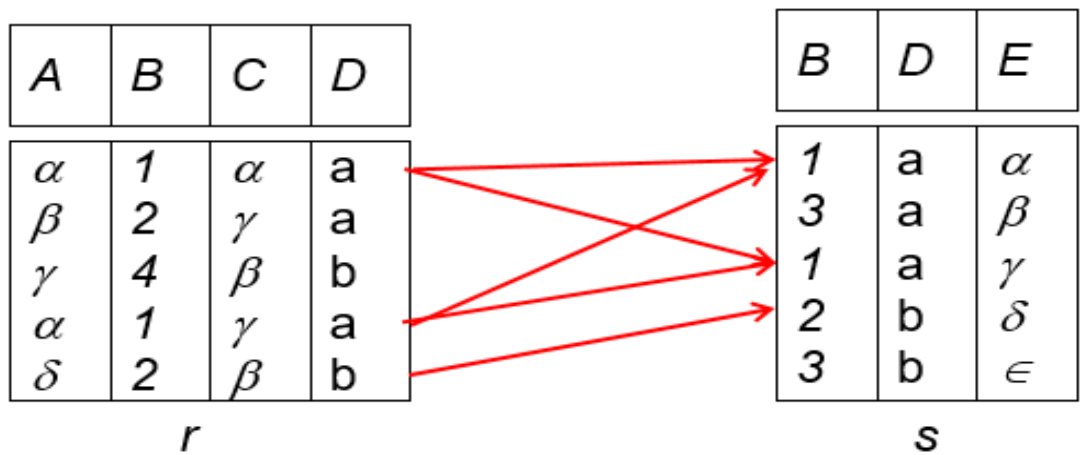
```
 $\rho(\text{CUST\_NAMES}, \Pi(\text{Customer\_Name})(\text{CUSTOMER}))$ 
```

Natural-Join Operation

- Binary operation that allows us to combine certain selections and a cartesian product into one operation
- Denoted by the join symbol $\bowtie$
- $r \bowtie s$
- Let r and s be relations on schemas R and S respectively. The result is a relation on schema $R \bowtie S$
- E.g., $R = (A, B, C, D)$ and $S = (E, B, D)$, then Result schema = (A, B, C, D, E)
- $r \bowtie s$ is defined as: $\prod_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B = s.B \wedge r.D = s.D} (r \times s))$
- Note: If $r(R)$ and $s(S)$ are relations without any attributes in common, that is, $R \cap S = \emptyset$, then $r \bowtie s = r \times s$

Natural-Join Operation

- Relations r, s:



<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

$r \bowtie s$

Assignment Operation

- Relational-algebra expression assign a part of a temporary relation variables using assignment operation ($\leftarrow$)
- Assignment must always be made to a temporary relation variable
- E.g., $\text{temp1} \leftarrow R \times S$, $\text{temp2} \leftarrow \sigma_{r.A1 = s.A1 \wedge r.A2 = s.A2 \wedge \dots \wedge r.An = s.An}(\text{temp1})$, then

$$\text{Result} = \prod_{R \cup S}(\text{temp2})$$
- The result to the right of the $\leftarrow$ is assigned to the relation variable on the left of the $\leftarrow$
- May use variable in subsequent expressions

Generalized Projection

- Extends the projection operation by allowing arithmetic functions to be used in the projection list
- $\prod_{F_1, F_2, \dots, F_n}(E)$ where E is any relational-algebra expression
- Each of $F_1, F_2, \dots, F_n$ are arithmetic expressions involving constants and attributes in the schema of E
- E.g., $\prod_{ID, name, dept\ name, salary \div 12}(instructor)$ gives the ID, name, dept name, and the monthly salary of each instructor

Aggregate Functions

- Takes a collection of values and returns a single value as a result
- avg: average value, min: minimum value, max: maximum value, sum: sum of values, count: number of values
- Aggregate operation in relational algebra $G_{G_1, G_2, \dots, G_n} F_1(A_1), F_2(A_2), \dots, F_n(A_n) (E)$
- Here, E is any relational-algebra expression
- $G_1, G_2, \dots, G_n$ is a list of attributes on which to group (can be empty), Each F_i is an aggregate function, Each A_i is an attribute name
- E.g.1, To find the average salary in each department

$$\text{dept_name } G_{\text{average(salary)}}(\text{instructor})$$
- E.g.2, To find out the sum of salaries of all instructors $G_{\text{sum(salary)}}(\text{instructor})$

Tuple Relational Calculus

- A non-procedural query language, where each query is of the form $\{t \mid P(t)\}$
- Set of all tuples t such that predicate P is true for t and t is a tuple variable
- $t \in r$ denotes that tuple t is in relation r and P is similar to predicate calculus
- Predicate Calculus includes
 - ✓ Set of attributes and constants
 - ✓ Set of comparison operators: (e.g., $<$, $\leq$, $=$, $\neq$, $>$, $\geq$)
 - ✓ Set of connectives: and ($\wedge$), or ($\vee$), not ($\neg$)
 - ✓ Implication ($\Rightarrow$): $x \Rightarrow y$, if x is true, then y is true means $x \Rightarrow y \equiv \neg x \vee y$
 - ✓ Set of quantifiers:
 - ❖ $\exists t \in r (Q(t)) \equiv$ "there exists" a tuple t in relation r
 - ❖ $\forall t \in r (Q(t)) \equiv Q$ is true "for all" tuples t in relation r

Tuple Relational Calculus

Schema for the University Database

- Instructor(ID,name,dept_name,salary)
- Teaches(ID,course_id,sec_id,semester,year)
- Takes(ID,course_id,sec_id,semester,year,grade)
- Section(course_id,sec_id,semester,year,building,room_no,time_slot_id)
- Student(ID,name,dept_name,tot_cred)
- Classroom(building,room_no,capacity)
- Department(dept_name,building,Budget)
- Course(course_id,title,dept_name,credits)

Relational Calculus:



Tuple Relational Calculus

Example Queries

- To find the ID, name, dept_name, salary for instructors whose salary is greater than \$80,000

$$\{t \mid t \in \text{instructor} \wedge t[\text{salary}] > 80000\}$$

- Notice that a relation on schema (ID, name, dept_name, salary) is implicitly defined by the query
- As in the previous query, but output only the ID attribute value

$$\{t \mid \exists s \in \text{instructor} (t[\text{ID}] = s[\text{ID}] \wedge s[\text{salary}] > 80000)\}$$

- Notice that a relation on schema (ID) is implicitly defined by the query

Tuple Relational Calculus: Part-I:



Part-II:



Tuple Relational Calculus

Example Queries

- To find the names of all instructors whose department is in the Watson building

$$\{t \mid \exists s \in \text{instructor } (t[\text{name}] = s[\text{name}] \wedge \exists u \in \text{department } (u[\text{dept_name}] = s[\text{dept_name}] \wedge u[\text{building}] = \text{"Watson"}))\}$$

- To find the set of all courses taught in the Fall 2009 semester, or in the Spring 2010 semester, or both

$$\{t \mid \exists s \in \text{section } (t[\text{course_id}] = s[\text{course_id}] \wedge s[\text{semester}] = \text{"Fall"} \wedge s[\text{year}] = 2009 \vee \exists u \in \text{section } (t[\text{course_id}] = u[\text{course_id}] \wedge u[\text{semester}] = \text{"Spring"} \wedge u[\text{year}] = 2010))\}$$

Tuple Relational Calculus

Example Queries

- To find the set of all courses taught in the Fall 2009 semester, and in the Spring 2010 semester

$$\{t \mid \exists s \in \text{section } (t[\text{course_id}] = s[\text{course_id}] \wedge s[\text{semester}] = \text{"Fall"} \wedge s[\text{year}] = 2009 \wedge \exists u \in \text{section } (t[\text{course_id}] = u[\text{course_id}] \wedge u[\text{semester}] = \text{"Spring"} \wedge u[\text{year}] = 2010))\}$$

- To find the set of all courses taught in the Fall 2009 semester, but not in the Spring 2010 semester

$$\{t \mid \exists s \in \text{section } (t[\text{course_id}] = s[\text{course_id}] \wedge s[\text{semester}] = \text{"Fall"} \wedge s[\text{year}] = 2009 \wedge \neg \exists u \in \text{section } (t[\text{course_id}] = u[\text{course_id}] \wedge u[\text{semester}] = \text{"Spring"} \wedge u[\text{year}] = 2010))\}$$

Tuple Relational Calculus

Universal Quantification

- To find all students who have taken all courses offered in the Biology department
- $\{t \mid \exists r \in \text{student } (t [\text{ID}] = r [\text{ID}]) \wedge (\forall u \in \text{course } (u [\text{dept_name}] = \text{"Biology"} \Rightarrow \exists s \in \text{takes } (t [\text{ID}] = s [\text{ID}] \wedge s [\text{course_id}] = u [\text{course_id}])))\}$

Safety of Expressions

- Possible to write tuple calculus expressions that generate infinite relations
- E.g., $\{t \mid \neg t \in r\}$ results in an infinite relation if the domain of any attribute of relation r is infinite
- To guard against the problem, restrict the set of allowable expressions to safe expressions

Tuple Relational Calculus

- An expression $\{t \mid P(t)\}$ is safe if every component of t appears in one of the relations, tuples, or constants that appear in P
- Note: This is more than just a syntax condition
- E.g., $\{t \mid t[A] = 5 \vee \text{true}\}$ is not safe --- it defines an infinite set with attribute values that do not appear in any relation or tuples or constants in P
- Consider again that query to find all students who have taken all courses offered in the Biology department

$$\{t \mid \exists r \in \text{student } (t[\text{ID}] = r[\text{ID}]) \wedge (\forall u \in \text{course } (u[\text{dept_name}] = \text{"Biology"} \Rightarrow \exists s \in \text{takes } (t[\text{ID}] = s[\text{ID}] \wedge s[\text{course_id}] = u[\text{course_id}]))))\}$$
- Without the existential quantification on student, the above query would be unsafe if the Biology department has not offered any courses

Domain Relational Calculus

- A nonprocedural query language equivalent in power to the tuple relational calculus
- Each query is an expression of the form: $\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$
- ✓ $x_1, x_2, \dots, x_n$ represent domain variables
- ✓ P represents a formula similar to that of the predicate calculus

Example Queries

- To find the ID, name, dept_name, salary for instructors whose salary is greater than \$80,000

$$\{ \langle i, n, d, s \rangle \mid \langle i, n, d, s \rangle \in \text{instructor} \wedge s > 80000 \}$$

- As in the previous query, but output only the ID attribute value

$$\{ \langle i \rangle \mid \langle i, n, d, s \rangle \in \text{instructor} \wedge s > 80000 \}$$

Domain Relational Calculus

Example Queries

- To find the names of all instructors whose department is in the Watson building

$$\{ \langle n \rangle \mid \exists i, d, s (\langle i, n, d, s \rangle \in \text{instructor} \wedge \exists b, a (\langle d, b, a \rangle \in \text{department} \wedge b = \text{"Watson"})) \}$$

- To find the set of all courses taught in the Fall 2009 semester, or in the Spring 2010 semester, or both

$$\{ \langle c \rangle \mid \exists a, s, y, b, r, t (\langle c, a, s, y, b, r, t \rangle \in \text{section} \wedge s = \text{"Fall"} \wedge y = 2009) \vee \exists a, s, y, b, r, t (\langle c, a, s, y, b, r, t \rangle \in \text{section} \wedge s = \text{"Spring"} \wedge y = 2010) \}$$

Domain Relational Calculus: Part-I:



Part-II:



Domain Relational Calculus

Example Queries

- This case can also be written as

$$\{ \langle c \rangle \mid \exists a, s, y, b, r, t \ (\langle c, a, s, y, b, r, t \rangle \in \text{section} \wedge ((s = \text{"Fall"} \wedge y = 2009) \vee (s = \text{"Spring"} \wedge y = 2010))) \}$$

- To find the set of all courses taught in the Fall 2009 semester, and in the Spring 2010 semester

$$\{ \langle c \rangle \mid \exists a, s, y, b, r, t \ (\langle c, a, s, y, b, r, t \rangle \in \text{section} \wedge s = \text{"Fall"} \wedge y = 2009) \\ \wedge \exists a, s, y, b, r, t \ (\langle c, a, s, y, b, r, t \rangle \in \text{section} \wedge s = \text{"Spring"} \wedge y = 2010) \}$$

Domain Relational Calculus

Safety of Expressions

- The expression: $\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$ is safe if all of the following hold:
- All values that appear in tuples of the expression are values from $\text{dom}(P)$ (that is, the values appear either in P or in a tuple of a relation mentioned in P)
- For every “there exists” subformula of the form $\exists x (P_1(x))$, the subformula is true if and only if there is a value of x in $\text{dom}(P_1)$ such that $P_1(x)$ is true
- For every “for all” subformula of the form $\forall x (P_1(x))$, the subformula is true if and only if $P_1(x)$ is true for all values x from $\text{dom}(P_1)$

Domain Relational Calculus

Universal Quantification

- To find all students who have taken all courses offered in the Biology department
- $$\{ \langle i \rangle \mid \exists n, d, tc (\langle i, n, d, tc \rangle \in \text{student} \wedge (\forall ci, ti, dn, cr (\langle ci, ti, dn, cr \rangle \in \text{course} \wedge dn = \text{"Biology"} \Rightarrow \exists si, se, y, g (\langle i, ci, si, se, y, g \rangle \in \text{takes}))) \}$$
- Note: Without the existential quantification on student, the above query would be unsafe if the Biology department has not offered any courses

Relational Algebra, Domain Relational Calculus, Tuple Relational Calculus

Summary

- Relational Algebra
- Schema for the University Database
- Select Operation (σ)
- Project Operation (π)
- Union Operation ($\cup$)
- Set Difference Operation ($-$)
- Set-Intersection Operation ($\cap$)
- Cartesian-Product Operation ($\times$)
- Rename Operation (ρ)
- Natural-Join Operation
- Assignment Operation
- Generalized Projection
- Aggregate Functions
- Tuple Relational Calculus
- Domain Relational Calculus

Relational Algebra, Relational Calculus – Fundamental Operations

Try Yourself

1. Write the queries in relational algebra, using the university schema.
 - a. Find the titles of courses in the Comp. Sci. department that have 3 credits.
 - b. Find the IDs of all students who were taught by an instructor named Einstein; make sure there are no duplicates in the result.
 - c. Find the highest salary of any instructor.
 - d. Find all instructors earning the highest salary (there may be more than one with the same salary).
 - e. Find the enrollment of each section that was offered in Autumn 2009.
 - f. Find the maximum enrollment, across all sections, in Autumn 2009.
 - g. Find the sections that had the maximum enrollment in Autumn 2009.

Relational Algebra, Domain Relational Calculus, Tuple Relational

Calculus Try Yourself

2. Consider the employee database

employee (person name, street, city)

works (person name, company name, salary)

company (company name, city)

manages (person name, manager name)

Consider the relational database of employee, where the primary keys are underlined. Give an expression in the relational algebra to express each of the following queries:

- a. Find the names of all employees who live in the same city and on the same street as do their managers.

Relational Algebra, Domain Relational Calculus, Tuple Relational Calculus

Try Yourself

- b. Find the names of all employees in this database who do not work for “First Bank Corporation”.
- c. Find the names of all employees who earn more than every employee of “Small Bank Corporation”.
- d. Find all employees who work directly for “Jones”.
- e. Find all cities of residence of all employees who work directly for “Jones”.
- f. Find the name of the manager of the manager of “Jones”.
- g. Find those employees who earn more than all employees living in the city “Mumbai”.

Session 5 – KEYS-PRIMARY KEY, FOREIGN KEY